

ADVANCED SQL ANALYTICS ASSIGNMENT

SQL Questions Q301–Q350

```
-- =====
-- SECTION 7 – ADVANCED SQL ANALYTICS
-- =====

-- Q301. Display each customer with total orders, total spending and
-- rank of the customer based on total spending.
WITH customer_summary AS (
    SELECT
        c.customer_id,
        c.customer_name,
        COUNT(o.order_id) AS total_orders,
        SUM(o.total_amount) AS total_spending
    FROM customers c
    INNER JOIN orders o
    ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.customer_name
)
SELECT
    customer_id,
    customer_name,
    total_orders,
    total_spending,
    RANK() OVER (ORDER BY total_spending DESC) AS spending_rank
FROM customer_summary;

-- Q302. Display the top 3 customers based on total spending.
WITH customer_summary AS (
    SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spending
    FROM customers c
    INNER JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.customer_name
),
ranked_customers AS (
    SELECT customer_id, customer_name, total_spending,
        DENSE_RANK() OVER (ORDER BY total_spending DESC) AS spending_rank
    FROM customer_summary
)
SELECT * FROM ranked_customers WHERE spending_rank <= 3;

-- Q303. Display the highest-value order of every customer.
WITH ranked_orders AS (
    SELECT o.order_id, o.customer_id, o.order_date, o.total_amount,
        ROW_NUMBER() OVER (
            PARTITION BY o.customer_id
            ORDER BY o.total_amount DESC
        ) AS rn
    FROM orders o
)
SELECT c.customer_name, ro.order_id, ro.order_date, ro.total_amount
FROM ranked_orders ro
INNER JOIN customers c ON ro.customer_id = c.customer_id
WHERE ro.rn = 1;

-- Q304. Display the latest order of every customer.
WITH latest_orders AS (
    SELECT o.order_id, o.customer_id, o.order_date, o.total_amount,
        ROW_NUMBER() OVER (
            PARTITION BY o.customer_id
            ORDER BY o.order_date DESC, o.order_id DESC
        ) AS rn
    FROM orders o
)
SELECT c.customer_name, lo.order_id, lo.order_date, lo.total_amount
FROM latest_orders lo
INNER JOIN customers c ON lo.customer_id = c.customer_id
WHERE lo.rn = 1;

-- Q305. Display products ranked by price within their category,
-- along with category name.
WITH ranked_products AS (
    SELECT p.product_id, p.product_name, p.category_id, p.price,
        RANK() OVER (
            PARTITION BY p.category_id
            ORDER BY p.price DESC
        ) AS price_rank
    FROM products p
)
SELECT rp.product_name, c.category_name, rp.price, rp.price_rank
FROM ranked_products rp
INNER JOIN categories c ON rp.category_id = c.category_id;
```

```

-- Q306. Display the top 2 most expensive products from every category.
WITH ranked_products AS (
    SELECT p.product_id, p.product_name, p.category_id, p.price,
           ROW_NUMBER() OVER (
               PARTITION BY p.category_id
               ORDER BY p.price DESC
           ) AS rn
    FROM products p
)
SELECT rp.product_name, c.category_name, rp.price
FROM ranked_products rp
INNER JOIN categories c ON rp.category_id = c.category_id
WHERE rp.rn <= 2;

-- Q307. Display brands whose average product price is greater than
-- the overall average product price.
WITH brand_average AS (
    SELECT brand_id, AVG(price) AS average_price
    FROM products
    GROUP BY brand_id
)
SELECT b.brand_name, ba.average_price
FROM brand_average ba
INNER JOIN brands b ON ba.brand_id = b.brand_id
WHERE ba.average_price > (SELECT AVG(price) FROM products);

-- Q308. Display categories having more products than the average
-- number of products per category.
WITH category_counts AS (
    SELECT category_id, COUNT(*) AS product_count
    FROM products
    GROUP BY category_id
)
SELECT c.category_name, cc.product_count
FROM category_counts cc
INNER JOIN categories c ON cc.category_id = c.category_id
WHERE cc.product_count > (SELECT AVG(product_count) FROM category_counts);

-- Q309. Display each product with its category average price
-- and difference from that average.
SELECT p.product_name, p.price,
       AVG(p.price) OVER (PARTITION BY p.category_id) AS category_average_price,
       p.price - AVG(p.price) OVER (PARTITION BY p.category_id) AS price_difference
FROM products p;

-- Q310. Display products priced above their category average.
SELECT p.product_name, p.price, c.category_name
FROM products p
INNER JOIN categories c ON p.category_id = c.category_id
WHERE p.price > (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.category_id = p.category_id
);

-- Q311. Display each customer with total spending and percentage
-- of the overall sales.
WITH customer_sales AS (
    SELECT customer_id, SUM(total_amount) AS total_spending
    FROM orders
    GROUP BY customer_id
)
SELECT c.customer_name, cs.total_spending,
       ROUND(cs.total_spending * 100.0 / SUM(cs.total_spending) OVER (), 2) AS sales_percentage
FROM customer_sales cs
INNER JOIN customers c ON cs.customer_id = c.customer_id;

-- Q312. Display each order with the previous order amount
-- and difference from previous order.
SELECT order_id, order_date, total_amount,
       LAG(total_amount) OVER (ORDER BY order_date, order_id) AS previous_amount,
       total_amount - LAG(total_amount) OVER (ORDER BY order_date, order_id) AS amount_difference
FROM orders;

-- Q313. Display each customer's orders with previous order amount
-- for the same customer.
SELECT customer_id, order_id, order_date, total_amount,
       LAG(total_amount) OVER (
           PARTITION BY customer_id
           ORDER BY order_date, order_id
       ) AS previous_order_amount
FROM orders;

-- Q314. Display customers whose latest order amount is greater
-- than their previous order amount.

```

```

WITH order_comparison AS (
    SELECT customer_id, order_id, order_date, total_amount,
           LAG(total_amount) OVER (
               PARTITION BY customer_id
               ORDER BY order_date, order_id
           ) AS previous_amount,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY order_date DESC, order_id DESC
           ) AS rn
    FROM orders
)
SELECT c.customer_name, oc.order_id, oc.total_amount, oc.previous_amount
FROM order_comparison oc
INNER JOIN customers c ON oc.customer_id = c.customer_id
WHERE oc.rn = 1 AND oc.total_amount > oc.previous_amount;

-- Q315. Display customers who have placed more orders than
-- the average orders per customer.
WITH customer_orders AS (
    SELECT customer_id, COUNT(*) AS total_orders
    FROM orders
    GROUP BY customer_id
)
SELECT c.customer_name, co.total_orders
FROM customer_orders co
INNER JOIN customers c ON co.customer_id = c.customer_id
WHERE co.total_orders > (SELECT AVG(total_orders) FROM customer_orders);

-- Q316. Display products with total quantity sold and rank
-- them based on quantity sold.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id
)
SELECT p.product_name, ps.total_quantity,
       RANK() OVER (ORDER BY ps.total_quantity DESC) AS sales_rank
FROM product_sales ps
INNER JOIN products p ON ps.product_id = p.product_id;

-- Q317. Display the top 5 products based on total quantity sold.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id
),
ranked_products AS (
    SELECT product_id, total_quantity,
           DENSE_RANK() OVER (ORDER BY total_quantity DESC) AS sales_rank
    FROM product_sales
)
SELECT p.product_name, rp.total_quantity, rp.sales_rank
FROM ranked_products rp
INNER JOIN products p ON rp.product_id = p.product_id
WHERE rp.sales_rank <= 5;

-- Q318. Display products whose total quantity sold is greater
-- than the average quantity sold per product.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id
)
SELECT p.product_name, ps.total_quantity
FROM product_sales ps
INNER JOIN products p ON ps.product_id = p.product_id
WHERE ps.total_quantity > (SELECT AVG(total_quantity) FROM product_sales);

-- Q319. Display each category with total stock and rank categories
-- based on total stock.
WITH category_stock AS (
    SELECT category_id, SUM(stock) AS total_stock
    FROM products
    GROUP BY category_id
)
SELECT c.category_name, cs.total_stock,
       RANK() OVER (ORDER BY cs.total_stock DESC) AS stock_rank
FROM category_stock cs
INNER JOIN categories c ON cs.category_id = c.category_id;

-- Q320. Display the category with the highest total stock.
WITH category_stock AS (
    SELECT category_id, SUM(stock) AS total_stock
    FROM products

```

```

        GROUP BY category_id
    ),
    ranked_categories AS (
        SELECT category_id, total_stock,
            RANK() OVER (ORDER BY total_stock DESC) AS stock_rank
        FROM category_stock
    )
    SELECT c.category_name, rc.total_stock
    FROM ranked_categories rc
    INNER JOIN categories c ON rc.category_id = c.category_id
    WHERE rc.stock_rank = 1;

-- Q321. Display payment methods with total number of payments
-- and total paid orders.
SELECT payment_method, COUNT(*) AS total_payments,
    SUM(CASE WHEN payment_status = 'Paid' THEN 1 ELSE 0 END) AS paid_payments
FROM payments
GROUP BY payment_method;

-- Q322. Display payment methods with their payment success rate.
SELECT payment_method, COUNT(*) AS total_payments,
    ROUND(
        SUM(CASE WHEN payment_status = 'Paid' THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
        2
    ) AS success_rate
FROM payments
GROUP BY payment_method;

-- Q323. Display customers who have at least one pending payment.
SELECT DISTINCT c.customer_name
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN payments p ON o.order_id = p.order_id
WHERE p.payment_status = 'Pending';

-- Q324. Display customers who have both Paid and Pending payments.
SELECT c.customer_name
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN payments p ON o.order_id = p.order_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(CASE WHEN p.payment_status = 'Paid' THEN 1 END) > 0
AND COUNT(CASE WHEN p.payment_status = 'Pending' THEN 1 END) > 0;

-- Q325. Display cities with customer count and total order amount.
SELECT c.city,
    COUNT(DISTINCT c.customer_id) AS customer_count,
    SUM(o.total_amount) AS total_sales
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city
HAVING SUM(o.total_amount) > 100000;

-- Q326. Display customers whose total spending is greater than
-- the average spending of customers from their same city.
WITH customer_city_sales AS (
    SELECT c.customer_id, c.customer_name, c.city,
        SUM(o.total_amount) AS total_spending
    FROM customers c
    INNER JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.customer_name, c.city
)
SELECT customer_name, city, total_spending
FROM customer_city_sales cs
WHERE total_spending > (
    SELECT AVG(cs2.total_spending)
    FROM customer_city_sales cs2
    WHERE cs2.city = cs.city
);

-- Q327. Display the most expensive product from every brand.
WITH ranked_products AS (
    SELECT product_id, product_name, brand_id, price,
        ROW_NUMBER() OVER (
            PARTITION BY brand_id
            ORDER BY price DESC, product_id
        ) AS rn
    FROM products
)
SELECT b.brand_name, rp.product_name, rp.price
FROM ranked_products rp
INNER JOIN brands b ON rp.brand_id = b.brand_id
WHERE rp.rn = 1;
-- Q328. Display brands whose highest-priced product is above 50000.

```

```

WITH brand_max_price AS (
    SELECT brand_id, MAX(price) AS highest_price
    FROM products
    GROUP BY brand_id
)
SELECT b.brand_name, bmp.highest_price
FROM brand_max_price bmp
INNER JOIN brands b ON bmp.brand_id = b.brand_id
WHERE bmp.highest_price > 50000;

-- Q329. Display products that have been ordered and whose total
-- quantity sold is greater than their available stock.
WITH product_sales AS (
    SELECT product_id, SUM(quantity) AS total_quantity
    FROM order_items
    GROUP BY product_id
)
SELECT p.product_name, p.stock, ps.total_quantity
FROM products p
INNER JOIN product_sales ps ON p.product_id = ps.product_id
WHERE ps.total_quantity > p.stock;

-- Q330. Display products that have never been ordered.
SELECT p.product_id, p.product_name, p.price
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
);

-- Q331. Display customers who have placed at least one order
-- above 50000.
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.total_amount > 50000
);

-- Q332. Display the percentage of each order's amount compared
-- with the customer's total spending.
SELECT order_id, customer_id, total_amount,
    ROUND(
        total_amount * 100.0 /
        SUM(total_amount) OVER (PARTITION BY customer_id),
        2
    ) AS customer_order_percentage
FROM orders;

-- Q333. Display each order and classify it as High, Medium or Low value.
SELECT order_id, total_amount,
    CASE
        WHEN total_amount >= 50000 THEN 'High Value'
        WHEN total_amount >= 10000 THEN 'Medium Value'
        ELSE 'Low Value'
    END AS order_category
FROM orders;

-- Q334. Count High, Medium and Low value orders.
SELECT
    CASE
        WHEN total_amount >= 50000 THEN 'High Value'
        WHEN total_amount >= 10000 THEN 'Medium Value'
        ELSE 'Low Value'
    END AS order_category,
    COUNT(*) AS total_orders
FROM orders
GROUP BY
    CASE
        WHEN total_amount >= 50000 THEN 'High Value'
        WHEN total_amount >= 10000 THEN 'Medium Value'
        ELSE 'Low Value'
    END;

-- Q335. Display products classified according to stock level.
SELECT product_name, stock,
    CASE
        WHEN stock >= 500 THEN 'Very High Stock'
        WHEN stock >= 100 THEN 'High Stock'
        WHEN stock >= 50 THEN 'Medium Stock'
        ELSE 'Low Stock'
    END AS stock_level
FROM products;

```

```

        END AS stock_category
FROM products;

-- Q336. Display customers and classify them based on total spending.
WITH customer_sales AS (
    SELECT customer_id, SUM(total_amount) AS total_spending
    FROM orders
    GROUP BY customer_id
)
SELECT c.customer_name, cs.total_spending,
    CASE
        WHEN cs.total_spending >= 100000 THEN 'Premium'
        WHEN cs.total_spending >= 50000 THEN 'Regular'
        ELSE 'Basic'
    END AS customer_type
FROM customer_sales cs
INNER JOIN customers c ON cs.customer_id = c.customer_id;

-- Q337. Display order status with total sales and rank each status
-- by total sales.
WITH status_sales AS (
    SELECT order_status, SUM(total_amount) AS total_sales
    FROM orders
    GROUP BY order_status
)
SELECT order_status, total_sales,
    RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM status_sales;

-- Q338. Display shipping cities with total orders and total sales.
SELECT s.city, COUNT(o.order_id) AS total_orders,
    SUM(o.total_amount) AS total_sales
FROM shipping s
INNER JOIN orders o ON s.shipping_id = o.shipping_id
GROUP BY s.city
HAVING COUNT(o.order_id) >= 5;

-- Q339. Display cities where average order amount is greater
-- than the overall average order amount.
SELECT c.city, AVG(o.total_amount) AS city_average_order
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.city
HAVING AVG(o.total_amount) > (SELECT AVG(total_amount) FROM orders);

-- Q340. Display the second-highest product price.
WITH ranked_products AS (
    SELECT product_id, product_name, price,
        DENSE_RANK() OVER (ORDER BY price DESC) AS price_rank
    FROM products
)
SELECT product_name, price
FROM ranked_products
WHERE price_rank = 2;

-- Q341. Display the second-highest order amount.
WITH ranked_orders AS (
    SELECT order_id, total_amount,
        DENSE_RANK() OVER (ORDER BY total_amount DESC) AS amount_rank
    FROM orders
)
SELECT order_id, total_amount
FROM ranked_orders
WHERE amount_rank = 2;

-- Q342. Display the top product by price from every category,
-- but only for categories having at least 3 products.
WITH category_counts AS (
    SELECT category_id, COUNT(*) AS product_count
    FROM products
    GROUP BY category_id
    HAVING COUNT(*) >= 3
),
ranked_products AS (
    SELECT product_id, product_name, category_id, price,
        ROW_NUMBER() OVER (
            PARTITION BY category_id
            ORDER BY price DESC, product_id
        ) AS rn
    FROM products
)
SELECT c.category_name, rp.product_name, rp.price
FROM ranked_products rp
INNER JOIN category_counts cc ON rp.category_id = cc.category_id

```

```

INNER JOIN categories c ON rp.category_id = c.category_id
WHERE rp.rn = 1;

-- Q343. Display customers with their first and last order dates.
SELECT c.customer_name,
       MIN(o.order_date) AS first_order_date,
       MAX(o.order_date) AS last_order_date
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;

-- Q344. Display customers whose first order amount is greater
-- than their average order amount.
WITH customer_orders AS (
    SELECT customer_id, order_id, order_date, total_amount,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY order_date, order_id
           ) AS rn,
           AVG(total_amount) OVER (
               PARTITION BY customer_id
           ) AS average_amount
    FROM orders
)
SELECT c.customer_name, co.order_id, co.total_amount, co.average_amount
FROM customer_orders co
INNER JOIN customers c ON co.customer_id = c.customer_id
WHERE co.rn = 1 AND co.total_amount > co.average_amount;

-- Q345. Display each product with total quantity sold,
-- total revenue and revenue rank.
WITH product_performance AS (
    SELECT p.product_id, p.product_name,
           SUM(oi.quantity) AS total_quantity,
           SUM(oi.quantity * oi.price) AS total_revenue
    FROM products p
    INNER JOIN order_items oi ON p.product_id = oi.product_id
    GROUP BY p.product_id, p.product_name
)
SELECT product_name, total_quantity, total_revenue,
       RANK() OVER (ORDER BY total_revenue DESC) AS revenue_rank
FROM product_performance;

-- Q346. Display the top 5 products based on revenue.
WITH product_revenue AS (
    SELECT product_id, SUM(quantity * price) AS revenue
    FROM order_items
    GROUP BY product_id
),
ranked_products AS (
    SELECT product_id, revenue,
           DENSE_RANK() OVER (ORDER BY revenue DESC) AS revenue_rank
    FROM product_revenue
)
SELECT p.product_name, rp.revenue, rp.revenue_rank
FROM ranked_products rp
INNER JOIN products p ON rp.product_id = p.product_id
WHERE rp.revenue_rank <= 5;

-- Q347. Display categories with total revenue and rank them.
WITH category_revenue AS (
    SELECT p.category_id, SUM(oi.quantity * oi.price) AS total_revenue
    FROM products p
    INNER JOIN order_items oi ON p.product_id = oi.product_id
    GROUP BY p.category_id
)
SELECT c.category_name, cr.total_revenue,
       RANK() OVER (ORDER BY cr.total_revenue DESC) AS revenue_rank
FROM category_revenue cr
INNER JOIN categories c ON cr.category_id = c.category_id;

-- Q348. Display customers whose total spending is in the top 10.
WITH customer_spending AS (
    SELECT customer_id, SUM(total_amount) AS total_spending
    FROM orders
    GROUP BY customer_id
),
ranked_customers AS (
    SELECT customer_id, total_spending,
           DENSE_RANK() OVER (ORDER BY total_spending DESC) AS spending_rank
    FROM customer_spending
)
SELECT c.customer_name, rc.total_spending, rc.spending_rank
FROM ranked_customers rc
INNER JOIN customers c ON rc.customer_id = c.customer_id

```

```
WHERE rc.spending_rank <= 10;
```

```
-- Q349. Compare total sales of Delivered, Pending and In Transit orders
-- using UNION ALL.
SELECT 'Delivered' AS status_type, COUNT(*) AS total_orders, SUM(total_amount) AS total_sales
FROM orders
WHERE order_status = 'Delivered'
UNION ALL
SELECT 'Pending' AS status_type, COUNT(*) AS total_orders, SUM(total_amount) AS total_sales
FROM orders
WHERE order_status = 'Pending'
UNION ALL
SELECT 'In Transit' AS status_type, COUNT(*) AS total_orders, SUM(total_amount) AS total_sales
FROM orders
WHERE order_status = 'In Transit';
```

```
-- Q350. Create a complete customer performance report with
-- orders, spending, average order value, rank and customer category.
WITH customer_performance AS (
    SELECT c.customer_id, c.customer_name, c.city,
           COUNT(o.order_id) AS total_orders,
           SUM(o.total_amount) AS total_spending,
           AVG(o.total_amount) AS average_order_value
    FROM customers c
    INNER JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.customer_name, c.city
),
ranked_customers AS (
    SELECT customer_id, customer_name, city, total_orders,
           total_spending, average_order_value,
           DENSE_RANK() OVER (ORDER BY total_spending DESC) AS spending_rank
    FROM customer_performance
)
SELECT customer_id, customer_name, city, total_orders,
       total_spending, average_order_value, spending_rank,
       CASE
           WHEN total_spending >= 100000 THEN 'Premium'
           WHEN total_spending >= 50000 THEN 'Regular'
           ELSE 'Basic'
       END AS customer_category
FROM ranked_customers
ORDER BY spending_rank;
```

```
-- =====
-- END OF SQL ASSIGNMENT
-- =====
-- Total Questions: 350
-- Database: ecommerce_db
-- Topics Covered:
-- WHERE CLAUSE
-- GROUP BY & HAVING
-- JOINS
-- SUBQUERIES
-- CTE
-- WINDOW FUNCTIONS
-- ADVANCED SQL ANALYTICS
-- =====
```